

Ethical Hacking Report

Ethical Hacking Report – Week 5

Target: `http://testphp.vulnweb.com` & `http://localhost:3000`

1. Reconnaissance (OSINT)

Nmap Scan: Port 80 (HTTP), 443 (HTTPS) open. Server: nginx/1.1.19.

WhatWeb: PHP 5.6.40, Apache.

Gobuster: Found `/admin/`, `/artists.php`, `/login.php` - potential attack surfaces.

2. SQL Injection Findings

Tool Used: SQLMap

Vulnerable Parameter: `artist` in `artists.php?artist=1`

Evidence: Successfully dumped database `acuart` and extracted `users` table containing admin hashes.

Fix Applied: Replaced dynamic SQL queries with

Prepared Statements (`db.execute()` with `?` placeholders) in `routes/api.js`.

3. CSRF (Cross-Site Request Forgery) Findings

Vulnerability: State-changing requests (POST /login) lacked anti-CSRF tokens.

Exploitation: Using Burp Suite, we replayed a login request without the token. Without protection, it would succeed.

Fix Applied: Integrated `csrf` middleware. Now all POST/PUT/DELETE requests require a valid signed CSRF token in the `x-csrf-token` header.

4. Conclusion

All identified vulnerabilities were successfully mitigated.

SQLi: **Fixed**

CSRF: **Fixed**

Rate Limiting: **Active**

Security Headers (HSTS/CSP): **Active**

Final File Structure (Week 5 Updated)

project now look like this:

```
secure-api/
├── .env
├── .gitignore
├── package.json
├── server.js      (UPDATED with CSRF & Cookie Parser)
├── db.js          (NEW)
├── logger.js
├── ETHICAL_HACKING_REPORT.md (NEW)
├── middleware/
│   ├── auth.js
│   └── rateLimiter.js
└── routes/
    └── api.js      (UPDATED with /users and /safe-users)
```

How to Test Everything:

1. Start your server:

```
bash
```

```
node server.js
```

2. Get a CSRF Token (using curl in a new terminal):

```
bash
```

```
curl http://localhost:3000/api/csrf-token -c cookies.txt
```

Copy the `csrfToken` from the JSON response.

3. Test the protected login (successful):

```
bash
```

```
curl -X POST http://localhost:3000/api/login \
-H "Content-Type: application/json" \
-H "x-csrf-token: YOUR_COPIED_TOKEN" \
```

```
-b cookies.txt \  
-d '{"username":"admin","password":"secret"}'
```

(Should return Login successful)

4. Test the CSRF block (remove the token):

bash

```
curl -X POST http://localhost:3000/api/login \  
-H "Content-Type: application/json" \  
-d '{"username":"admin","password":"secret"}'
```

(Should return Invalid CSRF token)

5. Test SQL Injection Fix:

Visit in browser:

<http://localhost:3000/api/safe-users?userId=1%20UNION%20SELECT%201,2,3>

(Should throw an error or return empty, not dump data).